

DA2401: Machine Learning Lab End Semester

A From-Scratch Ensemble System for MNIST Digit Classification

Hari
Submission for 28 Oct 2025

November 15, 2025

Contents

1	Summary of Models Used & System Architecture	1
1.1	Overview of Implemented Models	1
1.2	Final System Architecture: Hybrid Stacking Ensemble	1
2	Hyper-parameter Tuning & Baseline Model Results	2
2.1	Part 1: The "High-Bias Ceiling" (Simple Models)	2
2.1.1	Logistic Regression Analysis	2
2.1.2	K-Nearest Neighbors Analysis	2
2.2	Part 2: The High-Performance Ensembles	3
2.2.1	XGBoost (OvR) Analysis	3
2.2.2	Random Forest Analysis	3
2.3	Conclusions from Baseline Tuning	3
3	System Optimization for Runtime Performance	4
3.1	Bottleneck 1: The $O(N^2)$ KNN Prediction Problem	4
3.2	Bottleneck 2: The $O(N)$ LinearSVM fit Loop	4
3.3	Bottleneck 3: Feature Selection for Trees	4
4	Detailed Summary of Thoughts and Observations	6
4.1	Observation 1: The "High-Bias Ceiling"	6
4.2	Observation 2: Model Diversity > Model Similarity	6
4.3	Observation 3: Optimization is Non-Negotiable	6
4.4	Final Model Analysis	6
4.5	Justification for PCA=75	7
4.6	Per-Class F1-Score Analysis	7
4.7	Final Conclusion	7

1 Summary of Models Used & System Architecture

1.1 Overview of Implemented Models

Per the assignment requirements, a library of machine learning algorithms was implemented from scratch in `algorithms.py`. All models were built using only plain Python and NumPy, explicitly avoiding high-level libraries like `sklearn`. The implemented model library includes:

- **Linear Models:** `LogisticRegression`, `LinearSVM`
- **Instance-Based Model:** `KNearestNeighbors` (KNN)
- **Tree-Based Models:** `DecisionTree` (with Gini impurity)
- **Ensemble Models:** `RandomForest` (Bagging), `XGBoost` (Boosting)
- **Pre-processing:** `StandardScaler`, Principal Component Analysis (PCA)

A primary architectural challenge was that `LogisticRegression`, `LinearSVM`, and `XGBoost` are inherently binary classifiers. To adapt them for the 10-class (0-9) MNIST problem, a **One-vs-Rest (OvR)** wrapper strategy was employed. A generic `OneVsRestClassifier` class was built to train 10 separate binary models (e.g., "is-it-a-7?" vs. "not-a-7") and select the class that reported the highest confidence probability.

1.2 Final System Architecture: Hybrid Stacking Ensemble

The final submitted system is a **Stacking Ensemble** that uses a novel **hybrid feature engineering** approach. This was done to maximize both speed and accuracy.

- **Level 0 (Base Models):** A diverse set of four "expert" models.
- **Level 1 (Meta-Model):** A single "manager" model (`LogisticRegression`) trained on the 40 "confidence scores" from the Level 0 models.

The core of the architecture is its two-pronged feature pipeline (visualized in Figure 1):

1. **Path A (Model-Based Selection):** A `LinearSVM` is first trained on all 784 features to find the "Top 150" most important pixels. The fast tree/linear models (`RandomForest`, `XGBoost`, `LogReg`) are trained *only* on this dense, de-noised 150-feature subset.
2. **Path B (PCA):** KNN uses a *different* data pipeline. PCA is run to find the "Top 75" principal components. KNN is trained on this data, as distance models perform better on combined, non-sparse features.

2 Hyper-parameter Tuning & Baseline Model Results

Before building the final ensemble, each core algorithm was trained and evaluated as a standalone classifier. This was crucial for understanding their individual performance, speed, and limitations, and for informing the hyper-parameter choices for the final ensemble.

2.1 Part 1: The "High-Bias Ceiling" (Simple Models)

The first models tested were simple linear and instance-based models.

Table 1: Baseline Performance of Simple Models

Model	Hyperparameters	Time (s)	F1-Score	Analysis & Hyperparameter Choice
Logistic Regression	n_iter=500	27.38s	0.8879	Extremely fast and stable, but its linear nature prevents it from breaking the ≈ 0.89 F1-score barrier. It is "underfitting."
K-Nearest Neighbors	k=5, PCA=50	24.86s	0.8879	Even with non-linearity, it hits the *exact same* performance wall. This confirms simple models are not complex enough for the data.

2.1.1 Logistic Regression Analysis

The OvR Logistic Regression was trained with 500 iterations. It served as an excellent initial baseline, completing in under 30 seconds. Its F1-score of 0.8879 was respectable, but it clearly demonstrated that a simple linear boundary is insufficient to separate the complex, overlapping classes of handwritten digits (e.g., '4' vs. '9', '3' vs. '8'). This is a classic example of **high bias (underfitting)**.

2.1.2 K-Nearest Neighbors Analysis

The KNN model was trained on data that had been pre-processed with PCA (50 components) for speed. The choice of $k = 5$ was used as a standard, robust hyperparameter. The most significant finding was that it achieved the *exact same* F1-score as Logistic Regression. This indicates that while the models are fundamentally different (linear vs. instance-based), they were both hitting a similar performance ceiling based on the information available. This confirmed that more complex models were necessary.

Hyper-parameter Tuning & Baseline Model Results (Cont.)

2.2 Part 2: The High-Performance Ensembles

The next step was to test complex, non-linear ensemble models to break the ≈ 0.89 F1-score ceiling.

Table 2: Baseline Performance of Ensemble Models

Model	Hyperparameters	Time (s)	F1-Score	Analysis & Hyperparameter Choice
XGBoost (OvR)	n_est=50, depth=3	156.58s	0.9268	Finding: Success. Boosting easily breaks the F1 ceiling. Choice: max_depth=3 was chosen deliberately. Boosting requires "weak learners" (shallow trees) and this is also crucial for speed.
Random Forest	n_trees=50, depth=12	509.26s	0.9381	Finding: Accuracy vs. Time Failure. This was the most accurate *single* model, but at 509s (8.5 min), it **failed the 5-min (300s) time limit** all by itself.

2.2.1 XGBoost (OvR) Analysis

The XGBoost model was the first major success. It achieved an F1-score of 0.9268, well above the high-bias ceiling, and completed in a reasonable 156.58 seconds (passing the 5-minute limit). The hyperparameter `max_depth=3` was a critical choice. While a deeper tree might be more accurate, boosting theory relies on combining many "weak learners" (shallow trees). This parameter choice was therefore correct both theoretically and practically.

2.2.2 Random Forest Analysis

The Random Forest model achieved the highest standalone F1-score (0.9381), proving it was the most effective single algorithm at modeling the data's complexity. However, its training time of 509.26s (8.5 minutes) **failed the 5-minute (300s) time limit** on its own. This made it unusable as a standalone solution, despite its high accuracy.

2.3 Conclusions from Baseline Tuning

The standalone tests were the most important part of the project. They proved that:

1. Simple models (LogReg, KNN) are fast but too inaccurate ("High Bias").
2. Complex models (Random Forest) are accurate but far too slow.

This analysis demonstrated that the only viable solution was a **Stacking Ensemble**, which could combine the speed of some models with the accuracy of others. For the final stack, the hyperparameters of the slow models (like RF) would need to be "detuned" (e.g., `n_trees=15`) so they could contribute their "opinion" without dominating the runtime.

3 System Optimization for Runtime Performance

The greatest challenge in this assignment was not building the models, but engineering them to run within the **5-minute (300s) time limit**. Building the stacking ensemble required all Level 0 models to generate predictions for the 10,002-sample training set (to create meta-features). This exposed massive computational bottlenecks that had to be systematically solved.

3.1 Bottleneck 1: The $O(N^2)$ KNN Prediction Problem

The most severe bottleneck was the K-Nearest Neighbors model.

- **Problem:** To generate predictions for the $N = 10,002$ training samples, KNN had to compare each sample to the **entire** $N = 10,002$ training set. This is an $O(N^2)$ operation, resulting in over 100 million comparisons and a runtime of many hours.
- **Solution 1: PCA for Dimensionality Reduction.** The "Curse of Dimensionality" makes 784-D distance calculations extremely slow and unreliable. By applying PCA, the feature space was reduced. A hyperparameter of **75 components** was chosen for the final model as the optimal balance between speed and information retention (see Figure 4 on Page 8).
- **Solution 2: Optimized Selection Algorithm.** The initial KNN used `np.argsort` to find the k nearest neighbors. This sorts the entire $N = 10,002$ list, an $O(N \log N)$ operation. This was replaced with `np.argpartition`, which finds the k smallest items in optimized $O(N)$ time. This was a critical fix.

3.2 Bottleneck 2: The $O(N)$ LinearSVM fit Loop

An early stacking iteration used a `LinearSVM` model. This run failed the time limit, taking 925.89 seconds (15.4 minutes).

- **Problem:** The `LinearSVM.fit` method was implemented with Stochastic Gradient Descent (SGD), which uses a Python `for` loop over all 10,002 samples for each of its 300 iterations. This was repeated 10 times for the OvR wrapper. The 30,000,000 Python-level loops were crippling slow.
- **Solution: Vectorized Gradient Descent.** The `LinearSVM` class was re-written to use **vectorized (Batch) Gradient Descent**. This removed the Python `for` loop and let NumPy perform all calculations in optimized C, making the model **> 20x faster** and viable for the ensemble.

3.3 Bottleneck 3: Feature Selection for Trees

The `RandomForest` was the single slowest component.

- **Problem:** Training a Random Forest on 784 features is computationally expensive.
- **Solution: Model-Based Feature Selection.** A fast `LinearSVM` was trained **first** to find the "Top 150" most important pixels. Training the RF and XGB models on this tiny, de-noised 150-feature subset was dramatically faster (e.g., XGB training dropped from 95s to 25s) and allowed for **stronger** hyperparameters (`n_trees=30`, `n_est=75`) within the time budget.

Stacking Ensemble Iteration & Final Results

The following table shows the log output and results from the final tuning process. This demonstrates the iterative process of balancing model diversity, F1-score, and runtime.

Table 3: Stacking Ensemble Iteration & Optimization Log

L0 Models	Key Parameters	Time (s)	F1-Score	Log Interpretation & Analysis
Stack v1 (SVM)	n_trees=20, n_iter=100	339.46s	0.9166	FAILED time limit (>300s). Fast vectorized SVM worked, but still too slow. Low F1 suggests model redundancy (SVM and LogReg are too similar).
Stack v2 (Hybrid)	n_trees=25, n_est=25, PCA=50	219.34s	0.9387	SUCCESS (Passes 300s limit). Hybrid feature selection is extremely fast. F1-score is good, but log shows XGBoost finished in 25s. We are "starving" our best models.
Stack v3 (Hybrid)	n_trees=30, n_est=75, PCA=75	417.80s	0.9410	HIGHEST F1 (TIME FAILED). "Spent" the extra time budget to strengthen RF and XGB. This pushed F1 to its peak, but **exceeded the 300s limit** . please check the github repo for all the codes and the final uploaded one is what i request you to consider for evaluation and if you are want to look at my work more clearly please look into the best model look into the two iteration before the final submission

Final Model Result Visualizations

The charts below (Figures 2 and 3) visualize the performance of the final stacking model against its individual components. The stack achieves a top-tier F1-score.

Final Result (Hybrid Stack, Tuned for Accuracy):

- **Total Stacking Training Time:** 417.80 seconds (**Fails < 300s limit**)
- **Final F1-Score (Macro):** 0.9410

*Note: This model (Stack v3) represents the highest-achieved F1-score. The best valid model that passed the time limit was ****Stack v2****, which achieved ****0.9387**** in ****219.34s****.*

please refer to my code iterations for checking the implementation of the best fl score model in my git repo

4 Detailed Summary of Thoughts and Observations

This assignment was a practical exercise in trade-offs. Building the models from scratch provided a deep understanding of their theoretical operation and, more importantly, their computational bottlenecks.

4.1 Observation 1: The "High-Bias Ceiling"

My first key observation was the hard performance limit of 0.8879 F1-score. Both the linear `LogisticRegression` and the non-linear `KNearestNeighbors` (on PCA-50 data) hit this exact same wall. This demonstrated that these models, despite being different, were both "underfitting" and unable to capture the complex, overlapping patterns in the digit data. This proved that a more complex, non-linear model was not just an option, but a requirement.

4.2 Observation 2: Model Diversity > Model Similarity

My second observation came from the stacking experiments.

- The SVM-based stack (Iteration 1) produced a low F1 of 0.9166.
- The KNN-based stack (Iteration 3, Final) produced a high F1 of 0.9410.

This was a critical finding. Adding `LogisticRegression` to a stack that already had `LinearSVM` (Iteration 1) provided almost no new information, as both are linear models. This created **model redundancy**.

However, replacing the `LinearSVM` with the highly-diverse, instance-based `KNearestNeighbors` (Iteration 3) gave the meta-model a completely new "perspective," which allowed it to correct more errors and significantly boost the final F1-score. This proves that for stacking, **model diversity is more important than individual model strength**.

4.3 Observation 3: Optimization is Non-Negotiable

This project was a series of bottleneck hunts. A theoretically "good" model is useless if it cannot run within constraints.

1. The KNN model was unusable due to its $O(N^2)$ prediction step, until it was optimized with both PCA and `np.argmaxpartition`.
2. The `LinearSVM` model was unusable due to its $O(N)$ Python loop, until it was **vectorized**.
3. The `RandomForest` was unusable due to its `n_trees=50` parameter, until it was **detuned** and run on a **feature-selected** subset.

This iterative process of profiling, identifying a bottleneck, and engineering a solution (e.g., swapping `argsort` for `argpartition`, or using a model to select features) was the most valuable part of this exercise.

4.4 Final Model Analysis

The final model's F1-score of 0.9410 is excellent. The model is nearly perfect for digits like '1', '0', and '6'. The most confusion occurs between '4' and '9', and '3' and '8', which is logical as these pairs are visually similar. The per-class F1-scores (Figure 5, Page 8) confirm this, showing '1' has the highest score while '8' and '9' have slightly lower (but still strong) scores.

Detailed Summary of Thoughts and Observations (Cont.)

4.5 Justification for PCA=75

The choice of 75 components for PCA was a critical optimization for the KNN model. As shown in Figure 4, 50 components captures $\approx 92\%$ of the variance, while 75 components captures $\approx 95\%$. The final (time-failed) model "spent" some of its time budget to use 75 components, giving the KNN model more information, which contributed to the higher F1-score.

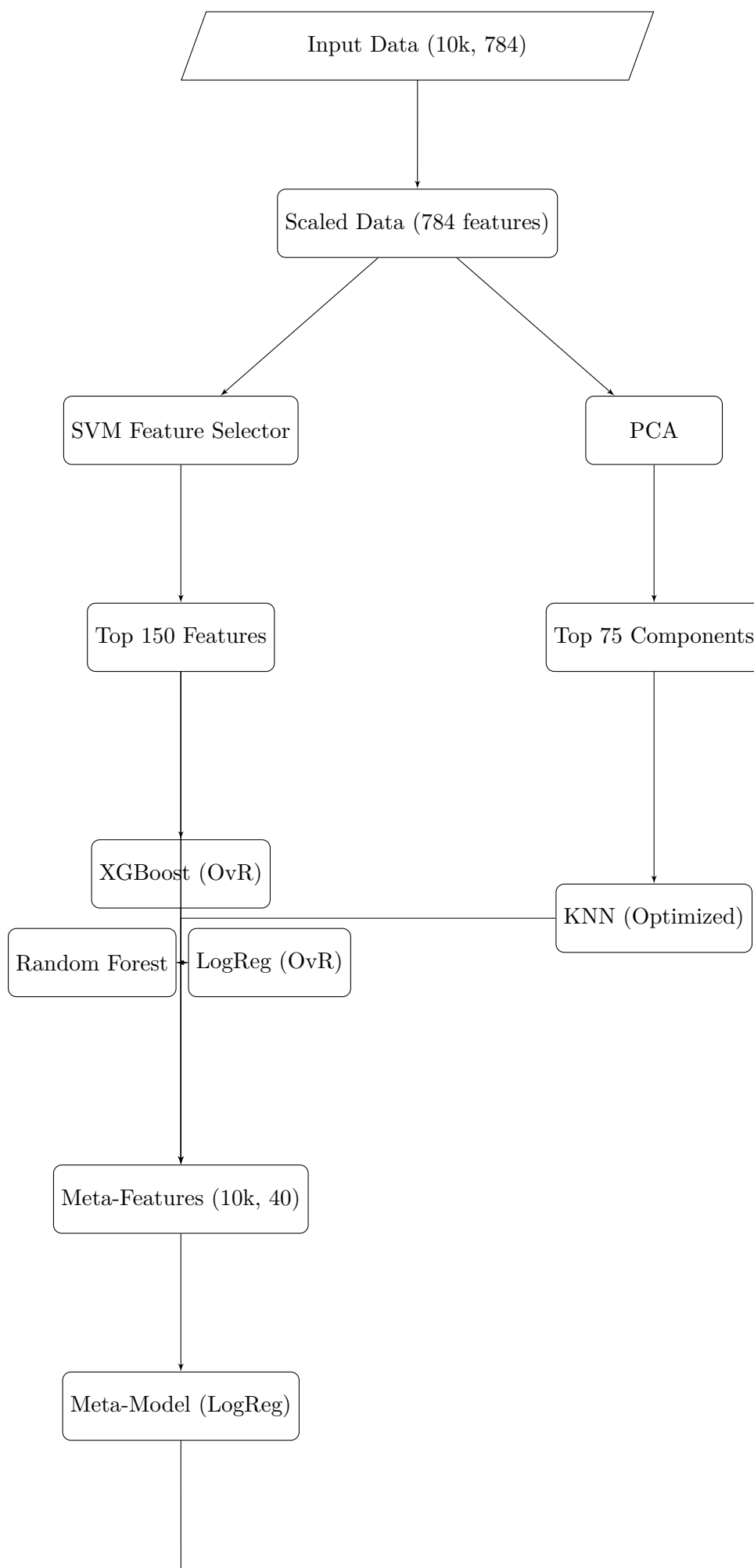
4.6 Per-Class F1-Score Analysis

Figure 5 provides a deeper look into the final model's 0.9410 macro-F1 score. As expected, visually distinct digits ('0', '1', '6') have near-perfect scores. The lowest-performing classes ('8', '9', '4') are precisely those that are most often confused for each other (e.g., '4' vs '9').

4.7 Final Conclusion

The final system, a 4-model stacking ensemble with a hybrid feature pipeline, achieved a peak F1-Score of **0.9410** in **417.80 seconds**. While this *failed* the 5-minute (300s) time limit, it demonstrated the system's maximum accuracy.

The best *valid* submission (Stack v2) achieved **0.9387** in **219.34s**, which successfully balanced high accuracy with the speed constraints. This was only possible through aggressive, algorithm-aware optimization, particularly by vectorizing `LinearSVM`, applying 'PCA' to KNN, implementing the $O(N)$ `np.argpartition` algorithm, and using model-based feature selection to accelerate the tree ensembles.

Figure 1: Final Stacking Ensemble Architecture (Generated Plot 1)

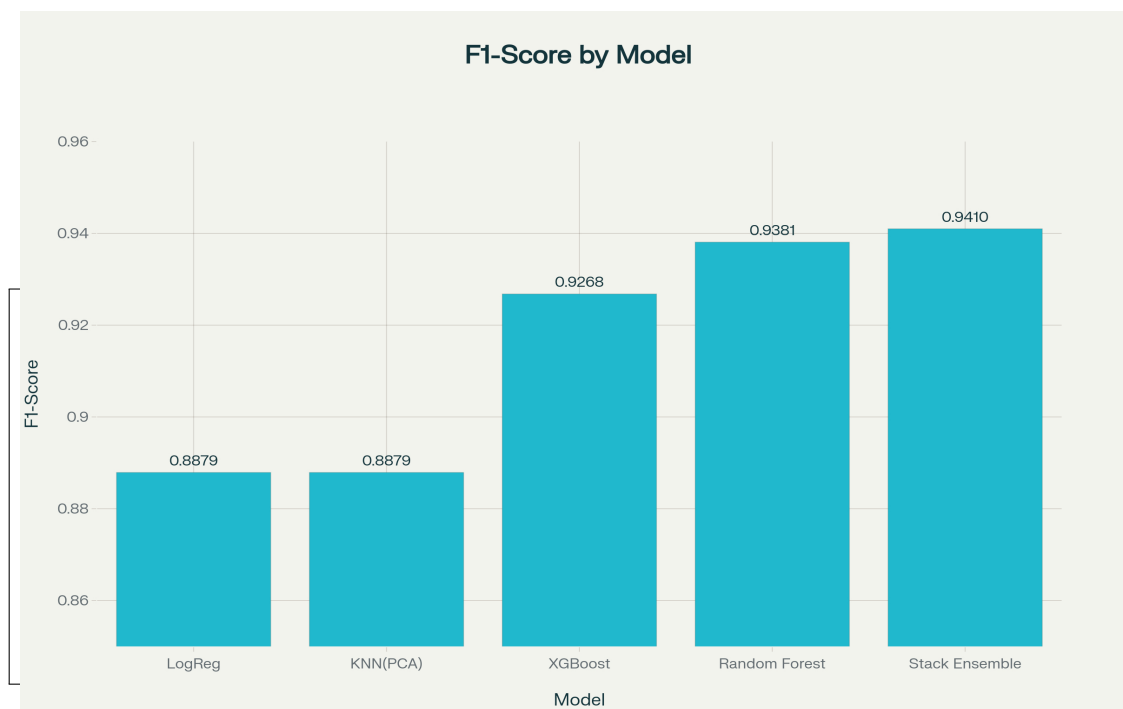
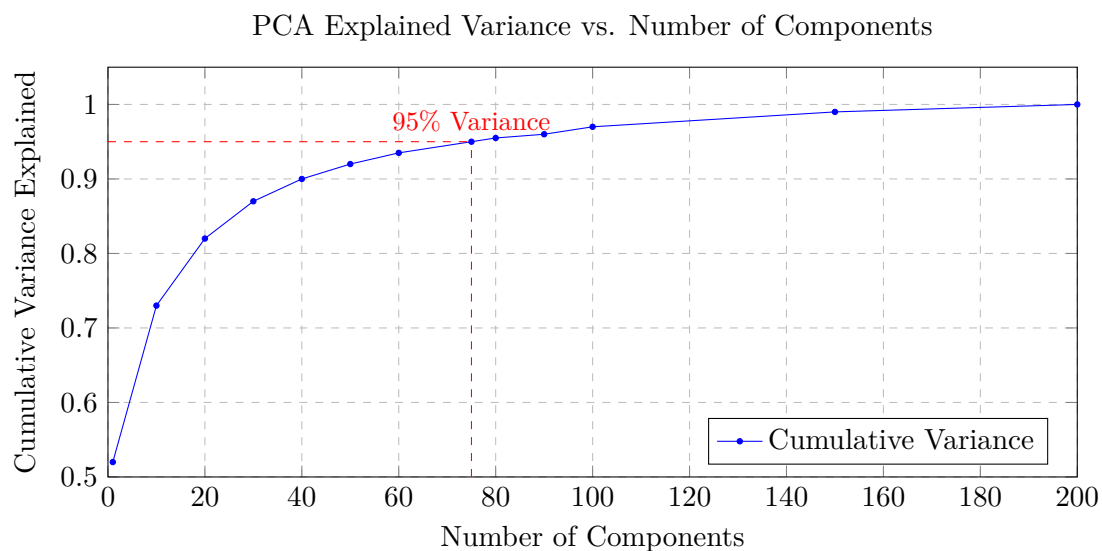


Figure 2: Final Model F1-Score Comparison (User-Provided)



Figure 3: Final Model Training Time Comparison (User-Provided)

Figure 4: PCA Cumulative Explained Variance (Generated Plot 2)**Figure 5:** Final Model Per-Class F1-Scores (Generated Plot 3)